```
const typeDefs = gql`

  type Movie {     #1
    id: Int
    name: String
    year: Int
  }

  type Query {     #2
    movie(movieId: Int!): Movie
    movies: [Movie]
  }
`;
```

#1 Movie type containing three fields: id, name, and year
#2 Query type defining all the queries allowed in our GraphQL server

The `typeDefs` object represents our GraphQL schema and contains two types:

- `Movie`—Represents a movie with the different fields it contains: `id`, `name`, and `year`.
- `Query`—Represents the queries allowed in our GraphQL server. The `movie` query represents a single movie and requires a mandatory `movieId`. There is also the query `movies`, which represents a list of movies.

To define the behavior of our queries, we define the `resolvers`:

```
const resolvers = {
  Query: {
    movies: (root, args, { dataSources }) => {
      return dataSources.movieApi.getMovies();     #1
    },
    movie: (root, { movieId }, { dataSources }) => {
      return dataSources.movieApi.getMovieById(movieId);     #2
    },
  },
}
```

#1 Calls the getMovies() to get all the movies
#2 Calls the getMovieById() and passes in the movieId to get a specific movie

The resolvers call the `getMovies` and `getMovieById` functions we defined in the class. To define our GraphQL server using Apollo, we create a new instance of `ApolloServer` and pass in our defined schema and the resolvers, as well as our data source. Once the GraphQL server is all set up, it’s time to start the server as shown in the following code: